Secure Exit

CO3-AT3-Assignment

← Back

SIMATS ENGINEERING

SIMATS Online Programming Lab

JAVA PROGRAMMING

Akileshwaran A
192511354RD

QUESTIONS

Q1
Banking Transaction System – Exception Handling
10 marks

Q2
Hospital Patient Registration – User-Defined Exception
10 marks

Q3
E-Commerce Order Processing – Built-in Exceptions
10 marks

Q4
Hospital Emergency System – Thread Priority
10 marks

Q5
Online Food Delivery – Multiple Threads and Execution Order
10 marks

5/5 answered

Q5 Online Food Delivery – Multiple Threads and Execution Order

10 marks

An online food-delivery application uses three threads: Order Processing, Payment Processing, and Delivery Assignment.

Develop a Java program in which these activities are represented by separate threads. Analyze the problem if all three threads are started independently.

Modify the program so that:
• Order processing completes before payment processing.
• Payment processing completes before delivery assignment.
• The program uses appropriate Java thread-management techniques.
• The output clearly shows the required execution order.

Explain the difference between simply starting all threads and explicitly controlling their execution order.
Sample Test Cases:
Input / Scenario Expected Result Analysis Focus
Order=Pizza; Payment=UPI; Delivery=Agent A Order → Payment → Delivery Normal execution
Order=Burger; Payment=Card; Delivery=Agent B Order → Payment → Delivery Concurrency / design
Payment=Failed Delivery assignment should not proceed Exception / boundary

Your Code

```
1 import java.util.*;
2 class Main{
3     public static void main(String[] args)throws InterruptedException{
4         Scanner sc=new Scanner(System.in);
```

```
        String input=sc.nextLine();
        if(sc.hasNextLine())
            input+=" " + sc.nextLine();
        String[] data=input.split(";");
        String order=data[0].split("=")[1].trim();
        String p=data[1].split("=")[1].trim();
        String d=data[2].split("=")[1].trim();
        oThread t1=new oThread(order);
        pThread t2=new pThread(p);
        dThread t3=new dThread(d);
        t1.start();
        t1.join();
        t2.start();
        t2.join();
        if(!p.equalsIgnoreCase("Failed")){
            t3.start();
            t3.join();
        }
        else{
            System.out.println("delivery assignment should not proceed")
        }
    }
}
class oThread extends Thread{
    String order;
    oThread(String order){
        this.order=order;
    }
    public void run(){
        System.out.println("order Processing: " + order);
    }
}
class pThread extends Thread{
    String p;
```

```
39     boolean success=true;
40     pThread(String p){
41         this.p=p;
42     }
43     public void run(){
44         if(p.equalsIgnoreCase("Failed")){
45             success=false;
46             System.out.println("payment Processing: "+ p);
47         }
48     }
49 }
50 class dThread extends Thread{
51     String d;
52     dThread(String d){
53         this.d=d;
54     }
55     public void run(){
56         System.out.println("delivery Assignment:"+ d);
57     }
58 }
```

Input

```
Order=Burger; Payment=Card;
Delivery=Agent B
```

Output

```
order Processing: Burger
delivery Assignment:Agent B
```

Submitted